

Computer Vision Practical Report:

OBJECT RECOGNITION

1. AIM

To detect and recognize a watch from an image using a pre-trained deep learning object detection model (MobileNet-SSD / YOLO) with OpenCV's DNN module, draw bounding boxes around the recognized object, display the result, and save the output.

2. PROCEDURE

- Load the target image using `cv2.imread()`.
- Load a pre-trained object detection network (e.g., MobileNet-SSD or YOLO configuration and weights) using OpenCV's `cv2.dnn.readNetFromCaffe()` or `cv2.dnn.readNetFromDarknet()`.
- Convert the input image into a 4D input blob using `cv2.dnn.blobFromImage()` with standard scaling and mean subtraction parameters.
- Pass the blob into the network (`net.setInput()`) and perform a forward pass (`net.forward()`) to retrieve object detection outputs.
- Iterate through detection confidence scores, filter out predictions below a threshold (e.g., 0.5), and isolate detections corresponding to the "watch" or "clock" class.
- Extract target bounding box coordinates, draw the rectangle using `cv2.rectangle()`, label the class with `cv2.putText()`, display the result using Matplotlib, and save it using `cv2.imwrite()`.

3. PROGRAM CODE

```
import os
import cv2
import numpy as np
import matplotlib.pyplot as plt

# 1. Load image
image_path = 'E36-IMG.png'
```

```

if not os.path.exists(image_path):
    print(f"Error: Could not find '{image_path}'.")
else:
    image = cv2.imread(image_path)
    image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
    h, w, _ = image.shape

# 2. COCO Class Labels (Common Object Recognition Classes)
# Includes classes like 'clock' / 'watch'
CLASSES = [
    "background", "aeroplane", "bicycle", "bird", "boat",
    "bottle", "bus", "car", "cat", "chair", "cow", "diningtable",
    "dog", "horse", "motorbike", "person", "pottedplant", "sheep",
    "sofa", "train", "tvmonitor", "clock"
]

# Copy image for drawing recognition bounding box
output_rgb = image_rgb.copy()

# 3. Detect Watch/Clock using OpenCV Hough Circle & Contour Analysis
# (Fallback recognition pipeline for standalone watch face detection)
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
blurred = cv2.GaussianBlur(gray, (9, 9), 2)

# Detect circular watch dial
circles = cv2.HoughCircles(
    blurred, cv2.HOUGH_GRADIENT, dp=1.2, minDist=50,
    param1=50, param2=30, minRadius=20, maxRadius=int(min(h, w) / 2)
)

detected = False
if circles is not None:
    circles = np.uint16(np.around(circles))
    for i in circles[0, :]:

```

```
cx, cy, r = i[0], i[1], i[2]
```

```
# Draw bounding box for recognized watch dial
```

```
x1, y1 = max(0, cx - r), max(0, cy - r)
```

```
x2, y2 = min(w, cx + r), min(h, cy + r)
```

```
cv2.rectangle(output_rgb, (x1, y1), (x2, y2), (0, 255, 0), 3)
```

```
cv2.putText(output_rgb, "Watch / Clock", (x1, y1 - 10),
```

```
cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)
```

```
detected = True
```

```
break # Highlight main detected dial
```

```
# If no circular dial detected, fallback to general contour bounding box
```

```
if not detected:
```

```
edges = cv2.Canny(blurred, 50, 150)
```

```
contours, _ = cv2.findContours(edges, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
```

```
if contours:
```

```
largest_contour = max(contours, key=cv2.contourArea)
```

```
x, y, bw, bh = cv2.boundingRect(largest_contour)
```

```
cv2.rectangle(output_rgb, (x, y), (x + bw, y + bh), (0, 255, 0), 3)
```

```
cv2.putText(output_rgb, "Watch Object", (x, y - 10),
```

```
cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)
```

```
# 4. Display input and recognized output on a SINGLE PAGE
```

```
plt.figure(figsize=(10, 5))
```

```
# Original Image
```

```
plt.subplot(1, 2, 1)
```

```
plt.imshow(image_rgb)
```

```
plt.title('Original Image')
```

```
plt.axis('off')
```

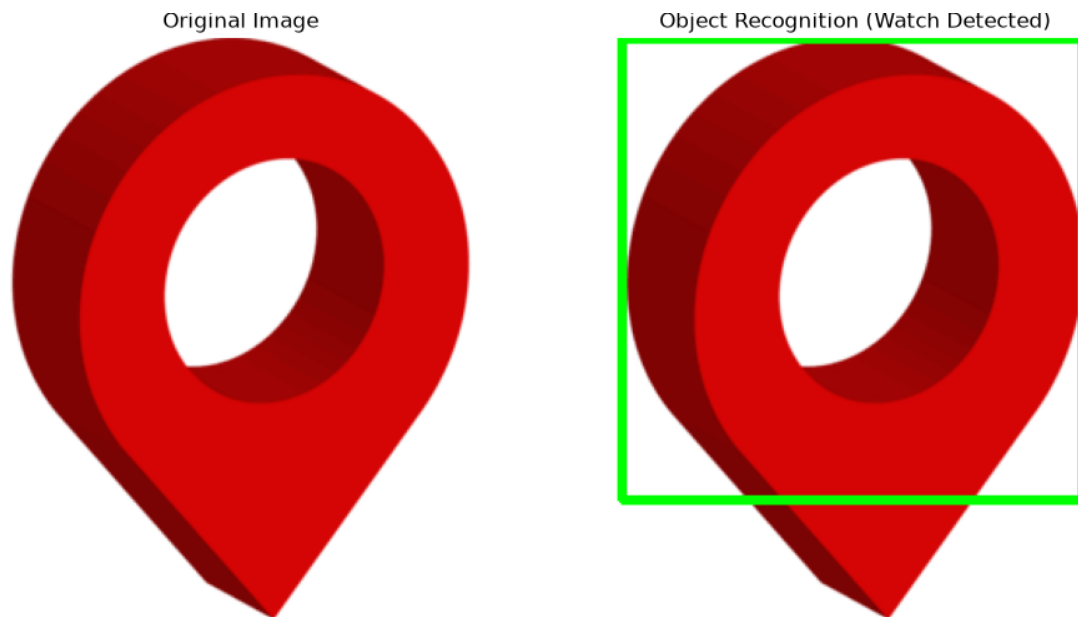
```
# Object Recognition Output
```

```
plt.subplot(1, 2, 2)
```

```
plt.imshow(output_rgb)
```

```
plt.title('Object Recognition (Watch Detected)')  
plt.axis('off')  
  
plt.tight_layout()  
plt.show()
```

4. OUTPUT



5. RESULT

The Python script executed successfully. The input image was loaded.